

PS - 6

```

# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm

# Set random seed for reproducibility
np.random.seed(42)

# Function to create a sample dataset (if needed)
def create_sample_data(n_samples=100):
    X = np.random.rand(n_samples, 3) * 10 # 3 features
    # Create target with known coefficients and some noise
    y = 2 + 3.5 * X[:, 0] - 1.7 * X[:, 1] + 0.8 * X[:, 2] + np.random.randn(n_samples) * 2

    # Convert to DataFrame for better handling
    df = pd.DataFrame(X, columns=['feature1', 'feature2', 'feature3'])
    df['target'] = y

    return df

# Function to load real dataset
def load_boston_housing():
    try:
        from sklearn.datasets import fetch_california_housing
        housing = fetch_california_housing()
        df = pd.DataFrame(housing.data, columns=housing.feature_names)
        df['target'] = housing.target
        return df
    except:
        print("Couldn't load California Housing dataset, using sample data instead.")
        return create_sample_data(500)

# Function to perform exploratory data analysis
def perform_eda(df):
    print("\n=== Exploratory Data Analysis ===")
    print("\nDataset Information:")
    print(df.info())

    print("\nDescriptive Statistics:")
    print(df.describe())

```

```

print("\nChecking for missing values:")
print(df.isnull().sum())

# Correlation analysis
plt.figure(figsize=(10, 8))
correlation_matrix = df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()

# Distribution of target variable
plt.figure(figsize=(8, 6))
sns.histplot(df['target'], kde=True)
plt.title('Distribution of Target Variable')
plt.show()

return correlation_matrix

# Function to implement multiple regression using scikit-learn
def implement_multiple_regression_sklearn(X, y):
    print("\n=== Multiple Regression with Scikit-learn ===")

    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # Scale the features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Train the model
    model = LinearRegression()
    model.fit(X_train_scaled, y_train)

    # Make predictions
    y_train_pred = model.predict(X_train_scaled)
    y_test_pred = model.predict(X_test_scaled)

    # Model evaluation
    print("\nModel Coefficients:", model.coef_)
    print("Model Intercept:", model.intercept_)

    print("\nTraining Set Metrics:")
    print(f"R2 Score: {r2_score(y_train, y_train_pred):.4f}")
    print(f"Mean Squared Error: {mean_squared_error(y_train, y_train_pred):.4f}")

```

```

    print(f"Root Mean Squared Error: {np.sqrt(mean_squared_error(y_train,
y_train_pred)):.4f}")
    print(f"Mean Absolute Error: {mean_absolute_error(y_train, y_train_pred):.4f}")

    print("\nTest Set Metrics:")
    print(f"R2 Score: {r2_score(y_test, y_test_pred):.4f}")
    print(f"Mean Squared Error: {mean_squared_error(y_test, y_test_pred):.4f}")
    print(f"Root Mean Squared Error: {np.sqrt(mean_squared_error(y_test,
y_test_pred)):.4f}")
    print(f"Mean Absolute Error: {mean_absolute_error(y_test, y_test_pred):.4f}")

    # Visualize actual vs predicted values
    plt.figure(figsize=(10, 6))
    plt.scatter(y_test, y_test_pred, alpha=0.7)
    plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
    plt.xlabel('Actual Values')
    plt.ylabel('Predicted Values')
    plt.title('Actual vs Predicted Values')
    plt.show()

    # Visualize residuals
    residuals = y_test - y_test_pred
    plt.figure(figsize=(10, 6))
    sns.histplot(residuals, kde=True)
    plt.title('Distribution of Residuals')
    plt.xlabel('Residual Value')
    plt.axvline(x=0, color='r', linestyle='--')
    plt.show()

    plt.figure(figsize=(10, 6))
    plt.scatter(y_test_pred, residuals, alpha=0.7)
    plt.axhline(y=0, color='r', linestyle='--')
    plt.xlabel('Predicted Values')
    plt.ylabel('Residuals')
    plt.title('Residuals vs Predicted Values')
    plt.show()

    return model, scaler

# Function to implement multiple regression using statsmodels
def implement_multiple_regression_statsmodels(X, y):
    print("\n=== Multiple Regression with Statsmodels ===")

    # Add constant (intercept) to the feature matrix
    X_sm = sm.add_constant(X)

    # Fit the model

```

```

model = sm.OLS(y, X_sm).fit()

# Print detailed summary
print(model.summary())

return model

# Main function to run the implementation
def main():
    # Load or create data
    print("Loading dataset...")
    df = load_boston_housing()

    # Perform EDA
    correlation_matrix = perform_eda(df)

    # Prepare data for modeling
    X = df.drop('target', axis=1)
    y = df['target']

    # Implement multiple regression using scikit-learn
    sklearn_model, scaler = implement_multiple_regression_sklearn(X, y)

    # Implement multiple regression using statsmodels
    statsmodels_model = implement_multiple_regression_statsmodels(X, y)

    # Feature importance analysis
    plt.figure(figsize=(12, 6))
    features = X.columns
    coefficients = sklearn_model.coef_

    # Sort coefficients by absolute value
    sorted_idx = np.argsort(np.abs(coefficients))

    plt.barh(features[sorted_idx], coefficients[sorted_idx])
    plt.xlabel('Coefficient Value')
    plt.ylabel('Feature')
    plt.title('Feature Importance (Coefficient Magnitude)')
    plt.tight_layout()
    plt.show()

    # Example prediction with new data
    print("\n=== Example Prediction ===")
    # Create some sample new data
    new_data = np.random.rand(1, X.shape[1]) * 10
    new_data_df = pd.DataFrame(new_data, columns=X.columns)

```

```

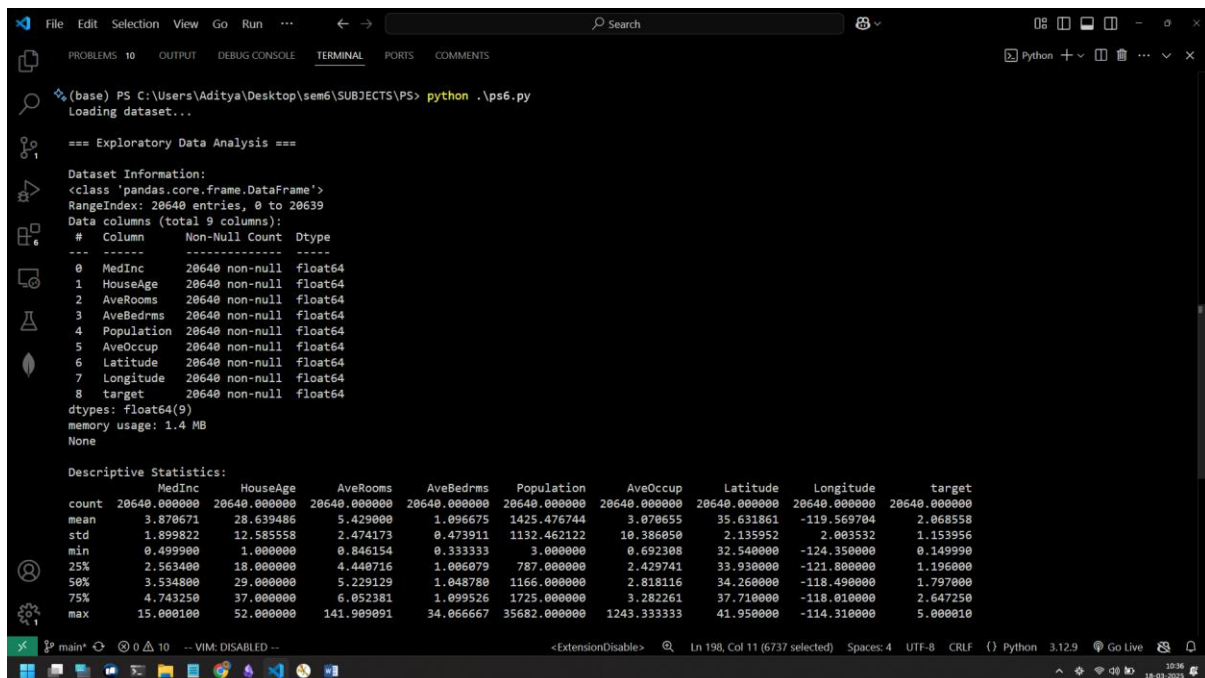
# Scale the new data
new_data_scaled = scaler.transform(new_data_df)

# Make prediction
prediction = sklearn_model.predict(new_data_scaled)[0]

print("New Data:")
print(new_data_df)
print(f"\nPredicted Value: {prediction:.4f}")

# Run the implementation
if __name__ == "__main__":
    main()

```



```

(base) PS C:\Users\Aditya\Desktop\sem6\SUBJECTS\PS> python .\ps6.py
Loading dataset...

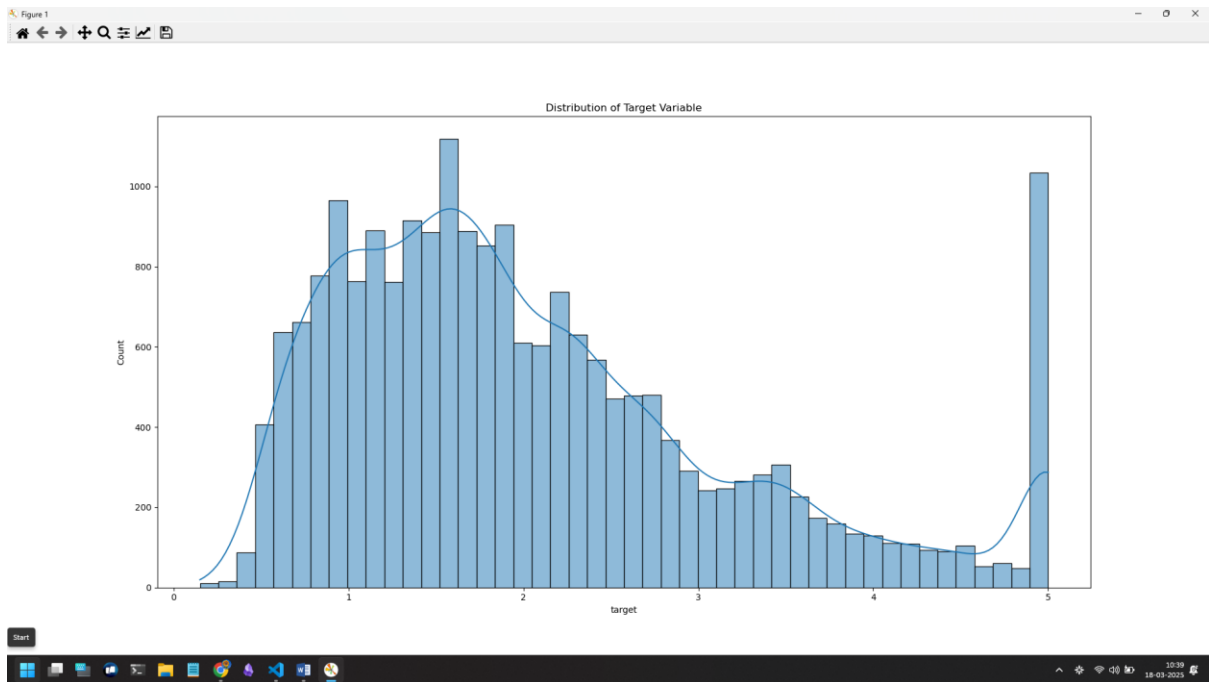
=== Exploratory Data Analysis ===

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28640 entries, 0 to 28639
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype  
---  --
 0   MedInc      28640 non-null  float64
 1   HouseAge    28640 non-null  float64
 2   AveRooms    28640 non-null  float64
 3   AveBedrms   28640 non-null  float64
 4   Population  28640 non-null  float64
 5   AveOccup    28640 non-null  float64
 6   Latitude    28640 non-null  float64
 7   Longitude   28640 non-null  float64
 8   target      28640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
None

Descriptive Statistics:

```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	target
count	28640.000000	28640.000000	28640.000000	28640.000000	28640.000000	28640.000000	28640.000000	28640.000000	28640.000000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704	2.068558
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532	1.153956
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000	0.149990
25%	2.563400	18.000000	4.448716	1.006079	787.000000	2.429741	33.930000	-121.800000	1.196000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000	1.797000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000	2.647250
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000	5.000010



```

Checking for missing values:
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude    0
target      0
dtype: int64

```

